

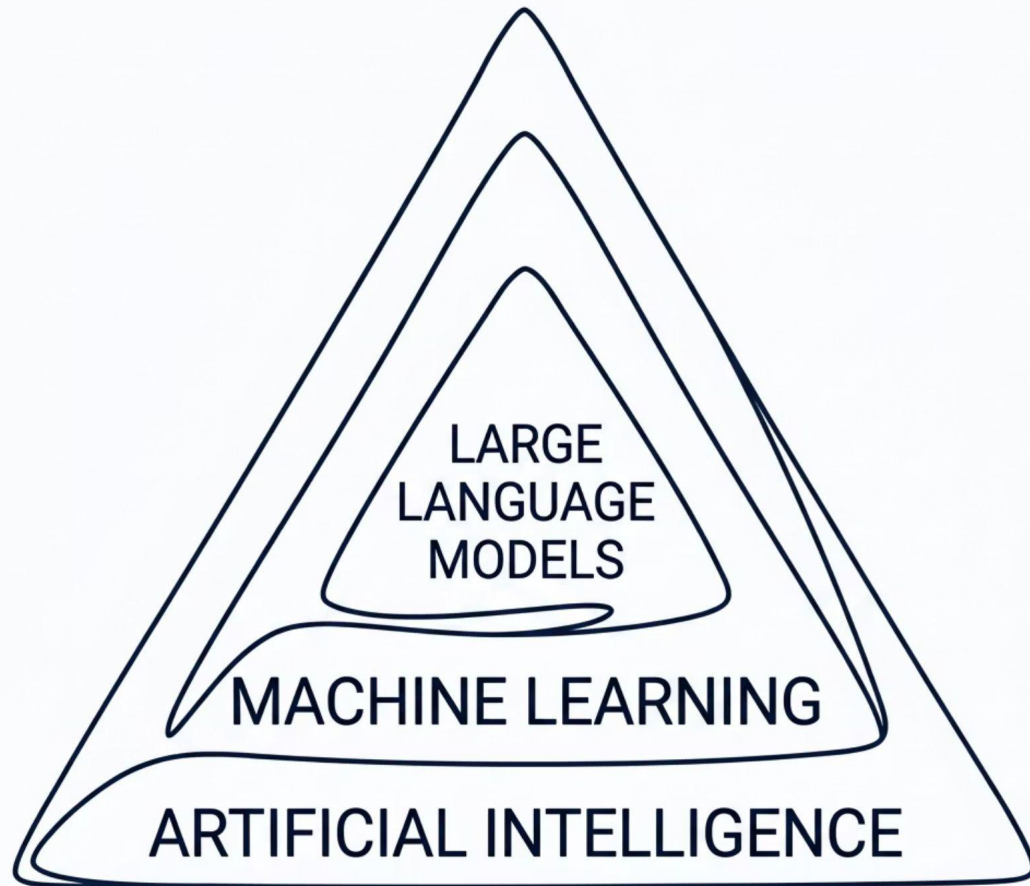


Large Language Models (LLMs)

Understanding How Modern Generative AI Works Behind the Scenes

FROM PROMPT TO RESPONSE

Where Do LLMs Fit?



LLMs Are the Engine of Modern GenAI

Large Language Models sit at the core of today's Generative AI landscape — purpose-built to understand and generate human language at scale.

Prominent LLM-powered applications:

Each application leverages an underlying language model to understand context and generate meaningful output.

ChatGPT
OpenAI GPT series

Gemini
Google DeepMind

Claude
Anthropic

Llama
Meta AI

What is a Large Language Model?

An LLM is a machine learning model trained on massive corpora of language data. It learns patterns, relationships, context, and structure — and can generate new, contextually relevant content.

Natural Language

Questions, instructions, conversations

Documents

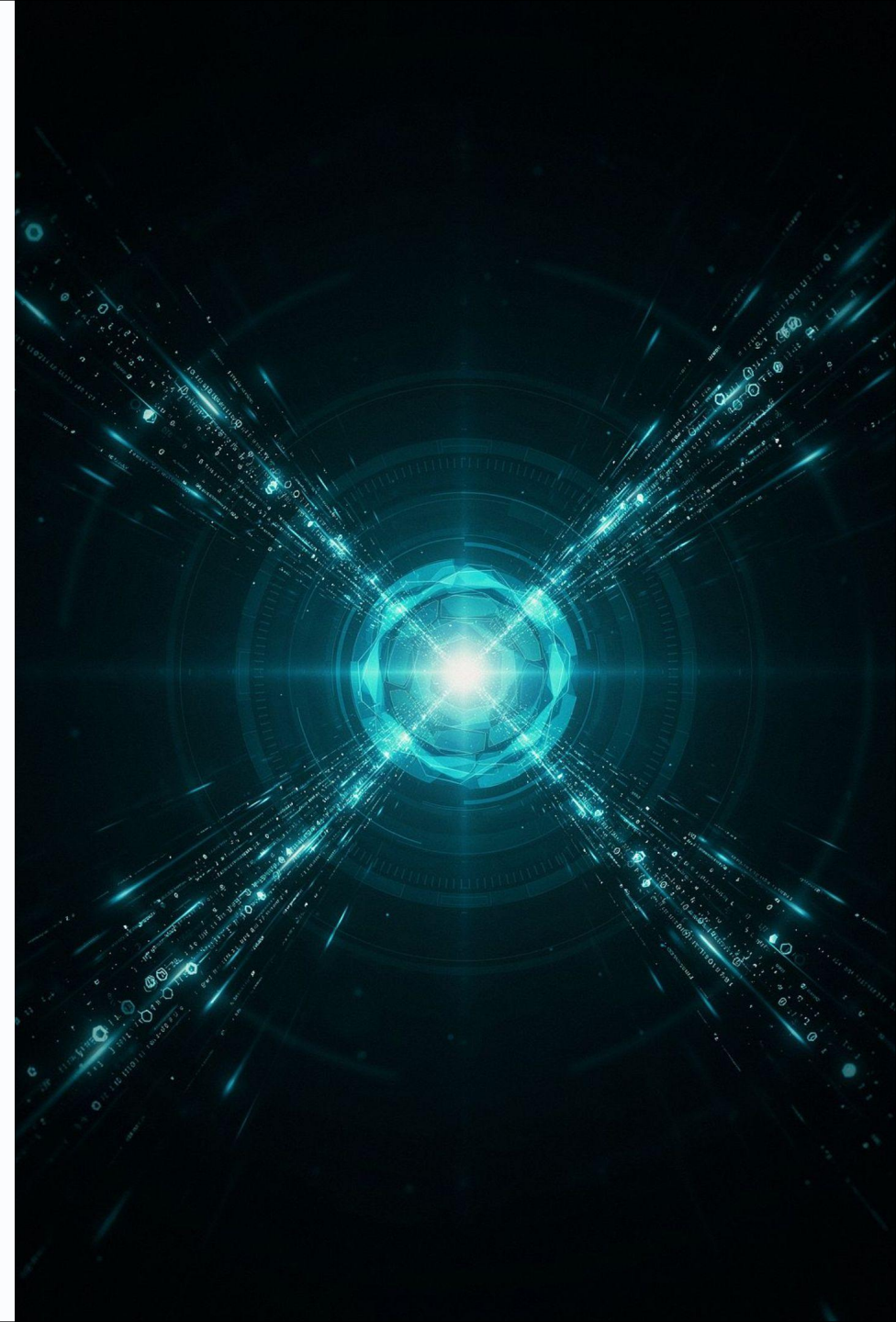
Reports, contracts, technical docs

Code

Python, SQL, JavaScript, APIs

Business Data

Analytics, schemas, business rules



Why Are LLMs Important?

Core Capabilities

- **Generate Text & Code**
Automated content and development assistance
- **Summarise & Analyse**
Distil insights from large document sets
- **Answer & Translate**
Q&A, multilingual support, report generation

Enterprise Use Cases

Banking

Risk analysis,
compliance, customer
service

Healthcare

Clinical notes, diagnosis
support

Retail

Product
recommendations,
sentiment

Data Engineering

Query generation,
pipeline documentation

LLM Application Flow

A language model usually sits inside an application that prepares the request and handles the result.



User and Interface

Collect the request and relevant application state



Prompt Assembly

Combine instructions, user input, and selected context



Model Invocation

Send the prepared input to the selected model provider



Response Handling

Check, format, and present the generated response

The application manages the workflow. The model performs inference on the input it receives.

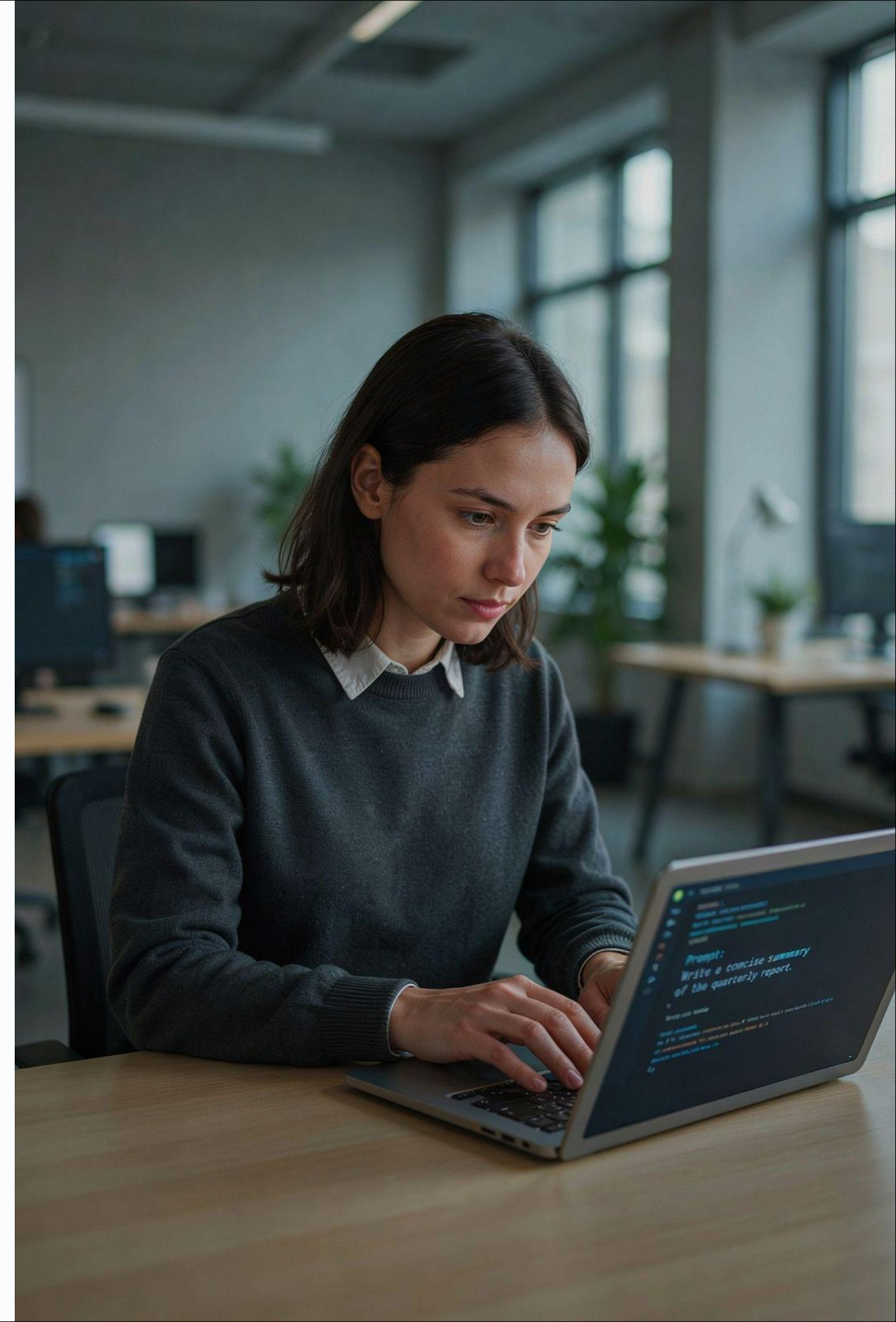
What Happens When We Enter a Prompt?

User Prompt: "Explain how solar panels generate electricity."

This single sentence triggers a sophisticated multi-stage pipeline inside the LLM. The model must answer four critical questions:

- 1 How does it understand the text?
- 2 How does it capture context?
- 3 How does it process language mathematically?
- 4 How does it generate the final response?

Each question corresponds to a distinct processing stage. Let us walk through the complete architecture, step by step.



Complete LLM Processing Flow

Every prompt travels through this end-to-end pipeline — repeated rapidly for each token in the generated response.



User Prompt

Tokenizer

Token IDs

Embeddings

❏ This entire pipeline executes in milliseconds per token — and repeats until the complete response is generated.



STEP 1

User Prompt

The pipeline begins the moment a user provides input. A prompt can take many forms:

- Question

"What is a product warranty?"

- Instruction

"Summarise this contract."

- Code Request

"Draft a Python function that validates order IDs."

- Document / Conversation

Multi-turn business dialogue

📄 Regardless of format, the input is collectively referred to as the **Prompt** — the starting point of all LLM processing.

Why Can't an LLM Directly Process Human Text?

The Core Challenge

An LLM is fundamentally a **mathematical model** — a neural network that operates entirely on numerical tensors. Human-readable text, as-is, cannot flow into these mathematical operations.

Before any processing can occur, raw text must be converted into a machine-processable numerical representation.

This conversion is handled by the first component in the pipeline:

Tokenizer



Tokenisation

Example Tokenisation

Breaking Text Into Tokens

The tokenizer receives the raw input and splits it into smaller units called **Tokens**. A token can represent:

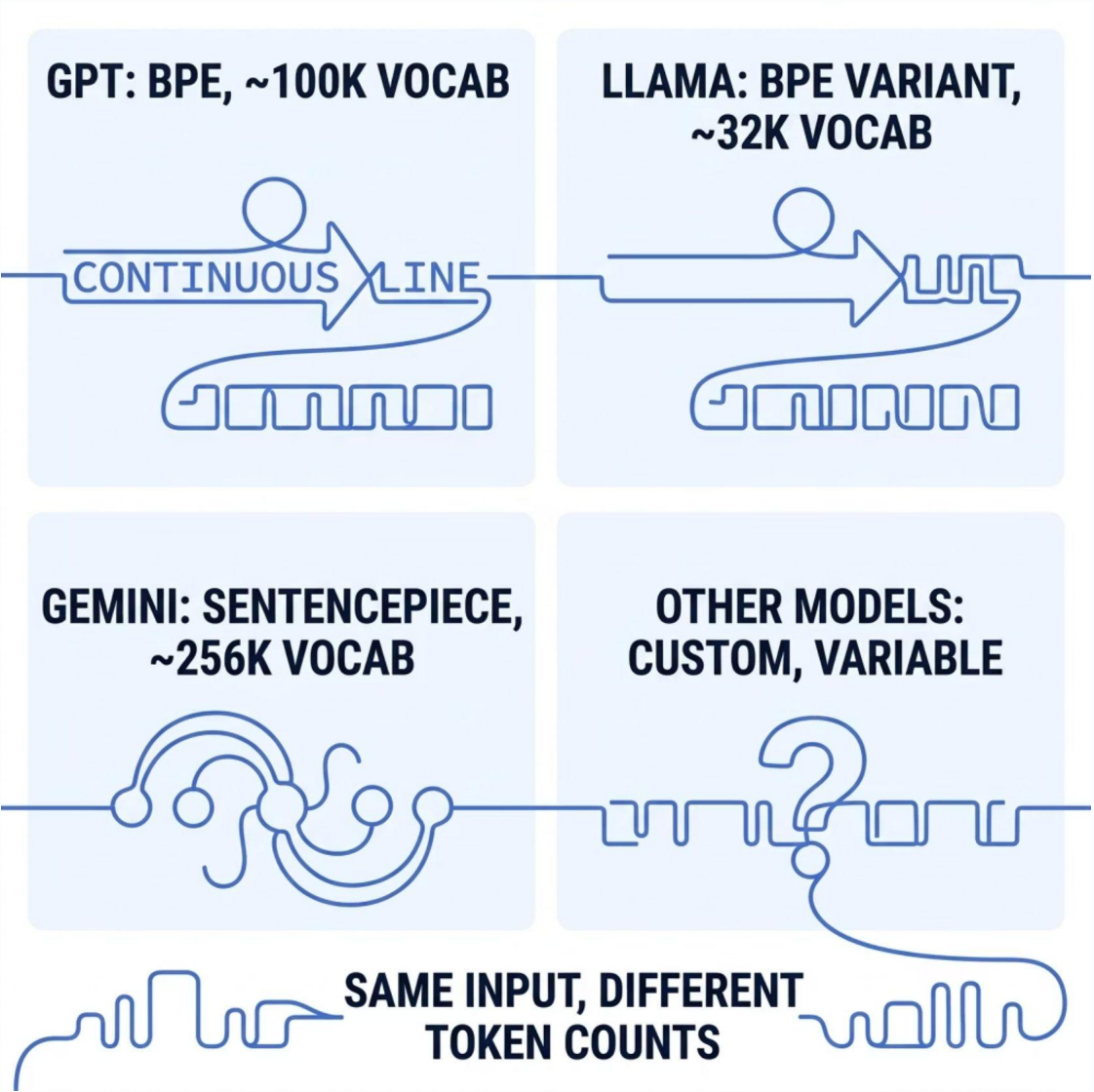
❑ **A Token ≠ A Word.** Tokenisation is model-specific and vocabulary-dependent.



Full Word "read", "data"	Sub-word "Data" → "Data", "bricks"	Punctuation " ", " ", " ", " ?"	Special Character Whitespace, newline patterns
-----------------------------	---------------------------------------	------------------------------------	---

Different Models, Different Tokenisers

Each LLM family uses its own tokenisation algorithm and vocabulary. The same prompt can yield different token counts across models.



Why Tokenisation Matters in Practice

- Context Window**
Maximum tokens the model can process per request
- API Pricing**
Most providers charge per input + output token
- Model Performance**
Token boundaries affect comprehension quality

Context and the Context Window

Context is the tokenized information available to the model during one generation request.

What Context Contains

- **Instructions**
System guidance and task requirements
- **Conversation History**
Selected prior messages needed for the current turn
- **Supplied Information**
Documents, tool results, or other task-specific data

How the Limit Affects the Application

Input Tokens

Instructions, history, and source material consume context.

Output Capacity

Generated tokens may also count toward the model limit.

Information Selection

Applications select relevant information and may trim old history.

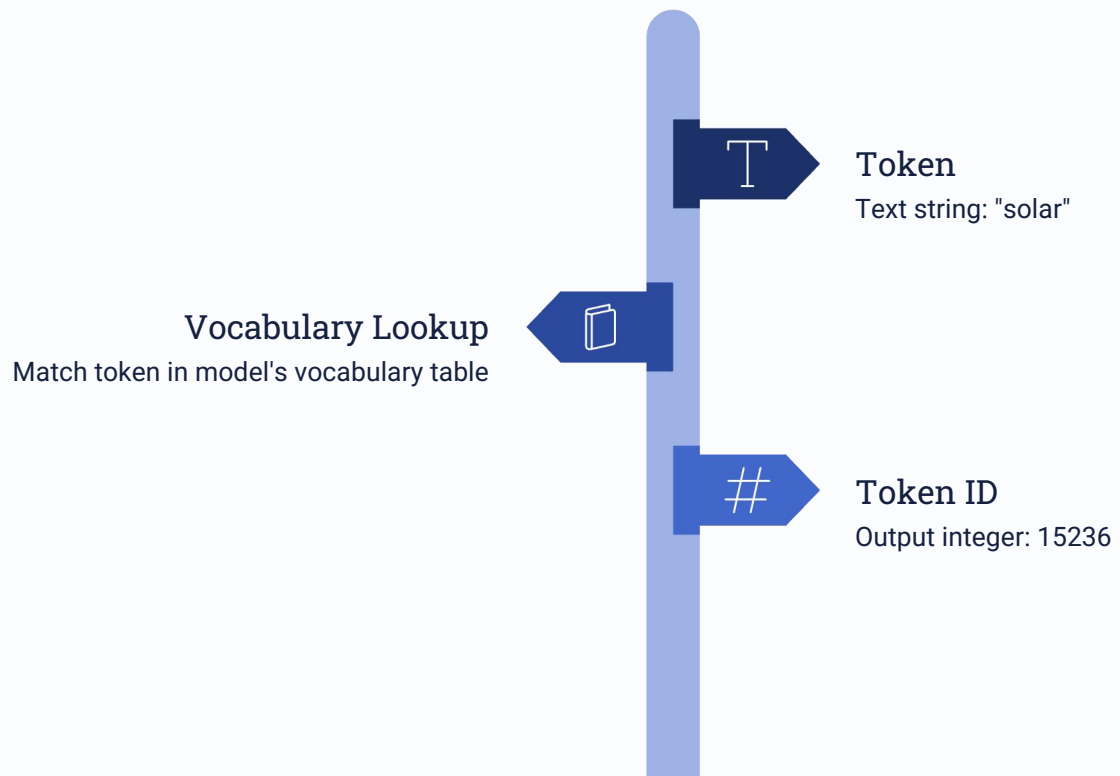
Request Memory

Context lasts for the request. It does not update model weights.

Token IDs

Mapping Tokens to Numbers

Once tokenisation is complete, every token is mapped to a unique numerical identifier — its **Token ID** — drawn from the model's fixed vocabulary lookup table.



Illustrative Examples

Token	Token ID (Illustrative)
Explain	3201
solar	15236
panels	2718
electricity	4024

⚠ Actual Token IDs are model-specific and vocabulary-dependent. These values are illustrative only.

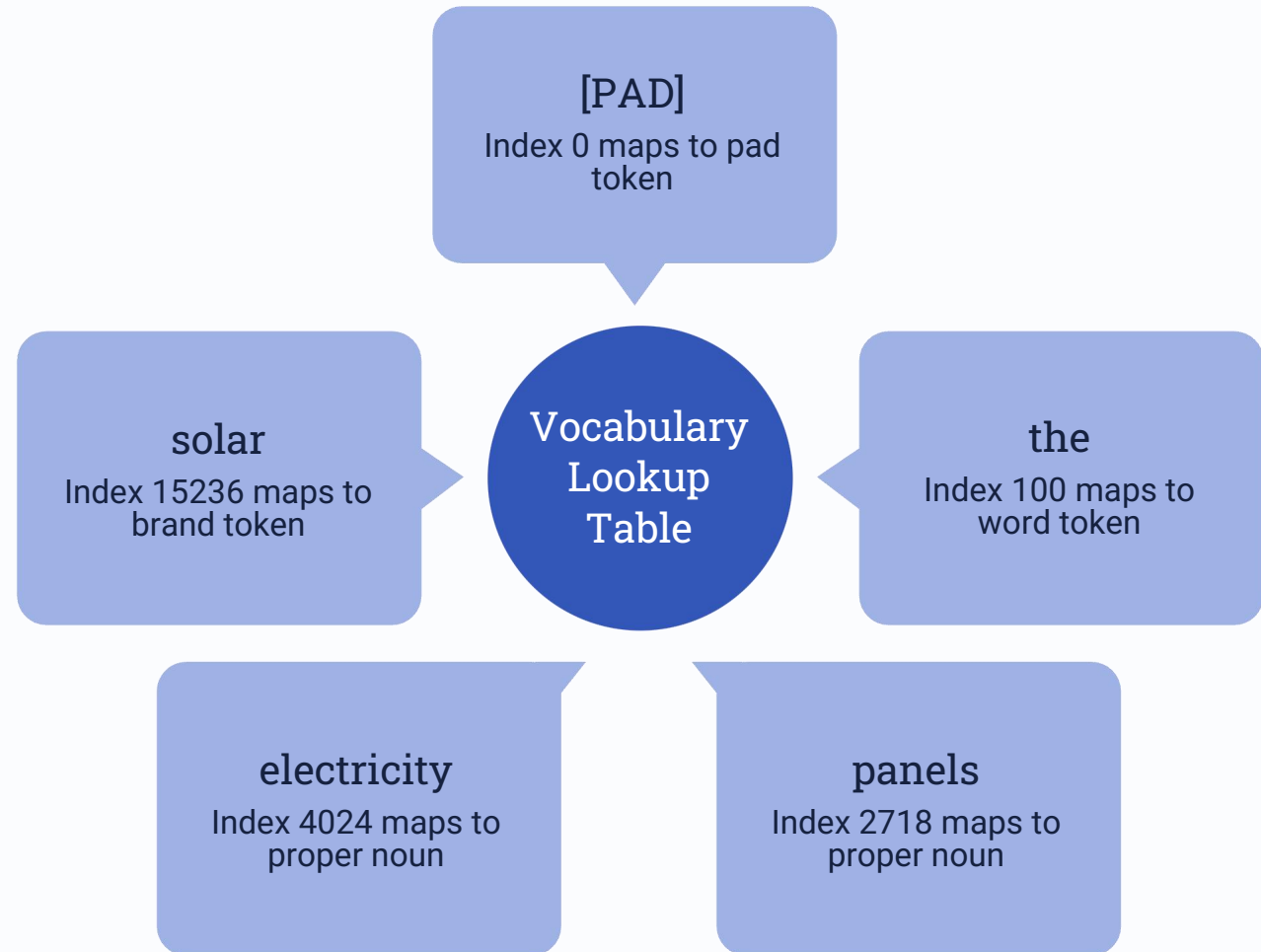
What Is a Token ID?

Token ID = Identifier Token ID $\neq$ Meaning

A Token ID is simply a positional index into the model's vocabulary table. The numerical value itself carries **no semantic weight**.

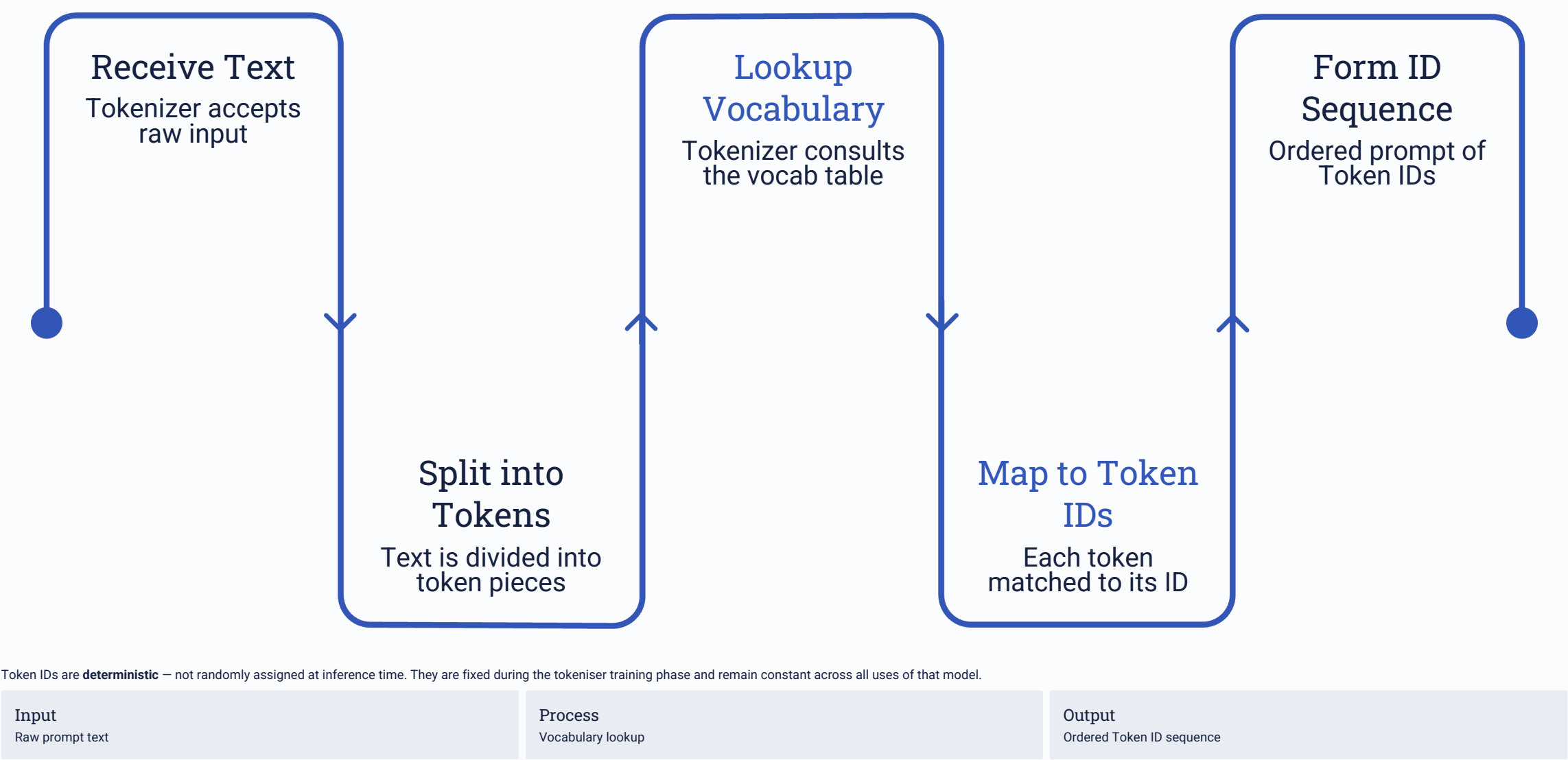
Token ID **100** and Token ID **5000** are not numerically related in meaning — they are purely lookup keys.

Vocabulary as a Lookup Table



Semantic meaning is captured later — by Embeddings, not by the ID value itself.

How Are Token IDs Generated?



What Happens After Token IDs?

We now have an ordered sequence of Token IDs. But a numerical index alone does not carry meaning. How does the model understand the *relationships* between these identifiers?

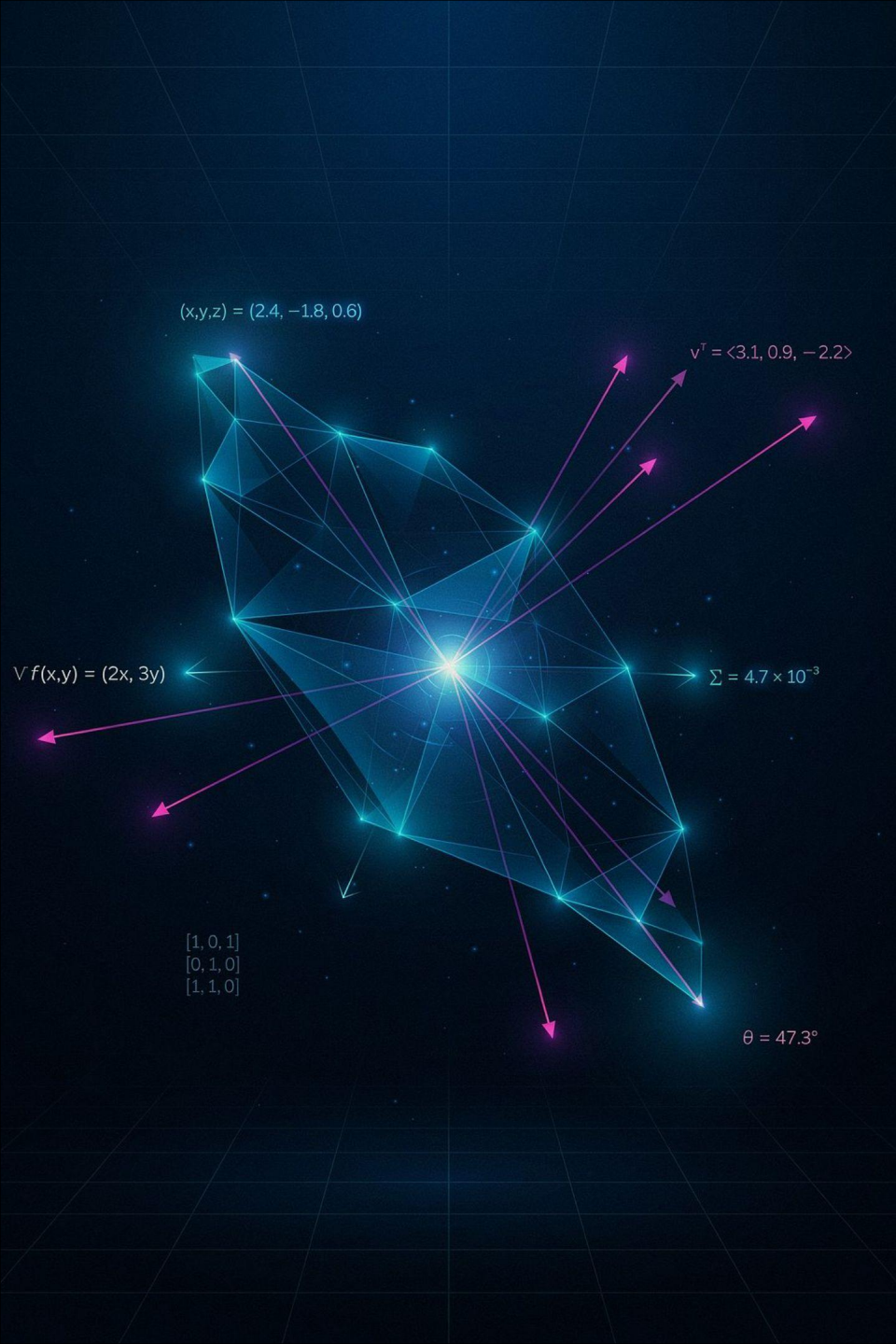
The answer lies in the next transformation stage:

Token

Token ID

Embedding Vector

Embeddings convert sparse identifiers into dense, information-rich numerical vectors that the neural network can meaningfully process.



What Is an Embedding?

From Index to Dense Vector

An **Embedding** is a high-dimensional numerical vector representation of a token. Unlike the Token ID (a sparse integer), an embedding is a dense array of floating-point values learned during model training.



Embedding Dimensions by Architecture

Model Family	Embedding Dimensions
GPT-2	768
GPT-3 / GPT-4	12,288
Llama 2 (7B)	4,096
Llama 3 (70B)	8,192

More dimensions allow the model to encode richer, more nuanced representations – at the cost of compute and memory.

Why Do We Need Embeddings?

Embeddings encode the statistical relationships learned across billions of training examples. Tokens that frequently co-occur in similar contexts will have numerically similar embedding vectors — enabling the model to leverage these learned patterns during inference.

Energy Context

These tokens appear in related technical contexts during training, so their embedding vectors reflect this co-occurrence.

Renewable Energy

Solar Panel

Customer

Transaction

Financial Domain Context

Similarly, financial tokens cluster together in embedding space due to their shared contextual usage.

Sunlight

Electricity

Revenue

Payment

Positional Information

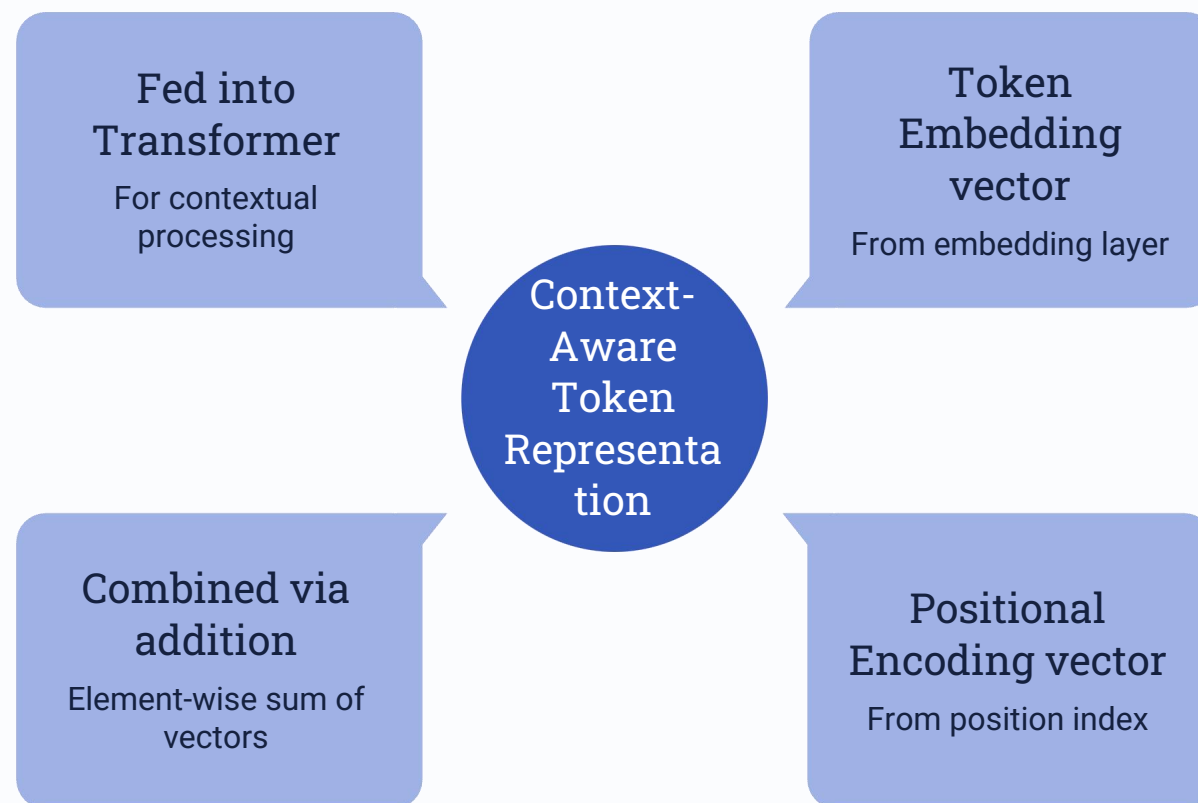
"Solar panels generate electricity efficiently."

"Electricity efficiently generates solar panels."

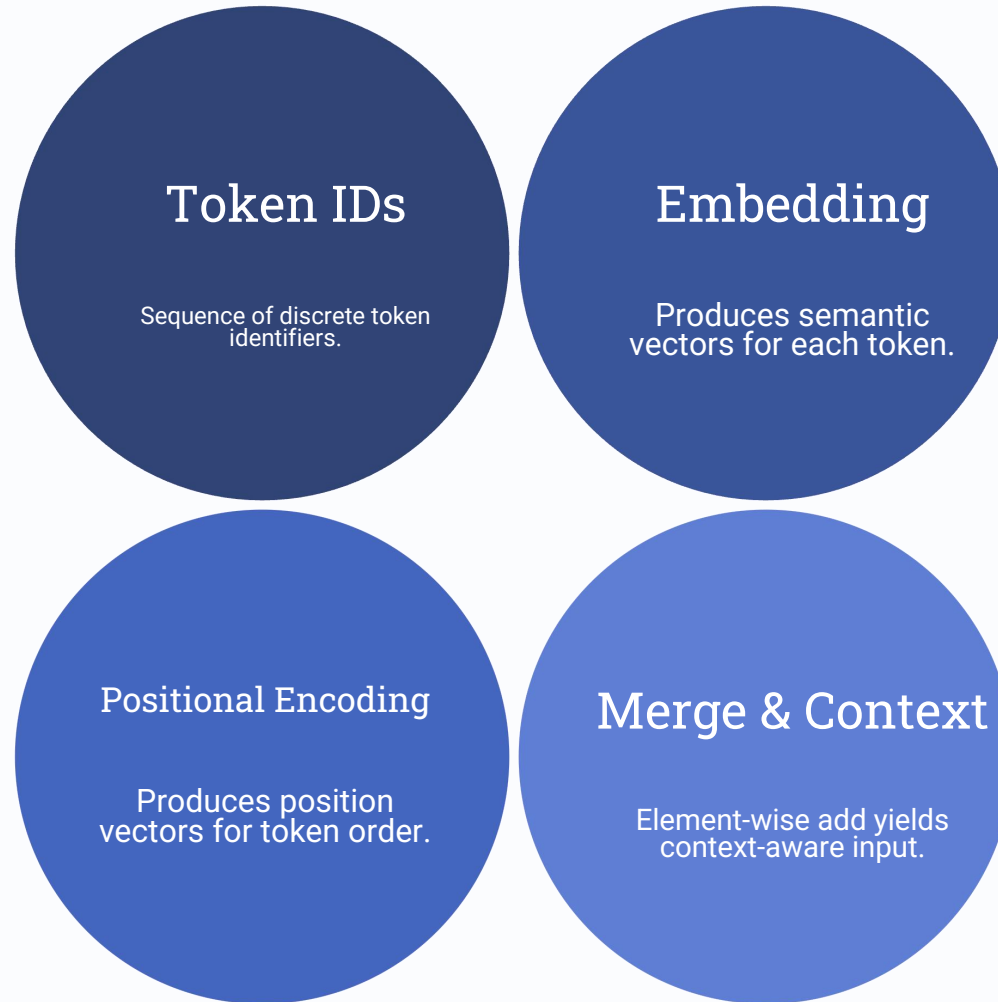
The tokens are identical. The sequence is different. The meaning is entirely different.

Embeddings encode *what* a token is — but not *where* it appears in the sequence. For a model to understand language, both pieces of information are essential.

This is where **Positional Encoding** comes in — a representation of each token's position within the input sequence, added to its embedding before Transformer processing.



Embeddings + Positional Information



What is this token?

Answered by the Embedding Vector

Where is this token?

Answered by Positional Encoding

Combined Output

Context-aware input representation — ready for Transformer processing

What Is a Transformer?

The **Transformer** is the core neural architecture underpinning most modern LLMs. Introduced in the landmark 2017 paper *"Attention Is All You Need"*, it processes the entire token sequence simultaneously — identifying relationships between all tokens in context.

GPT Family

OpenAI — decoder-only Transformer

Llama

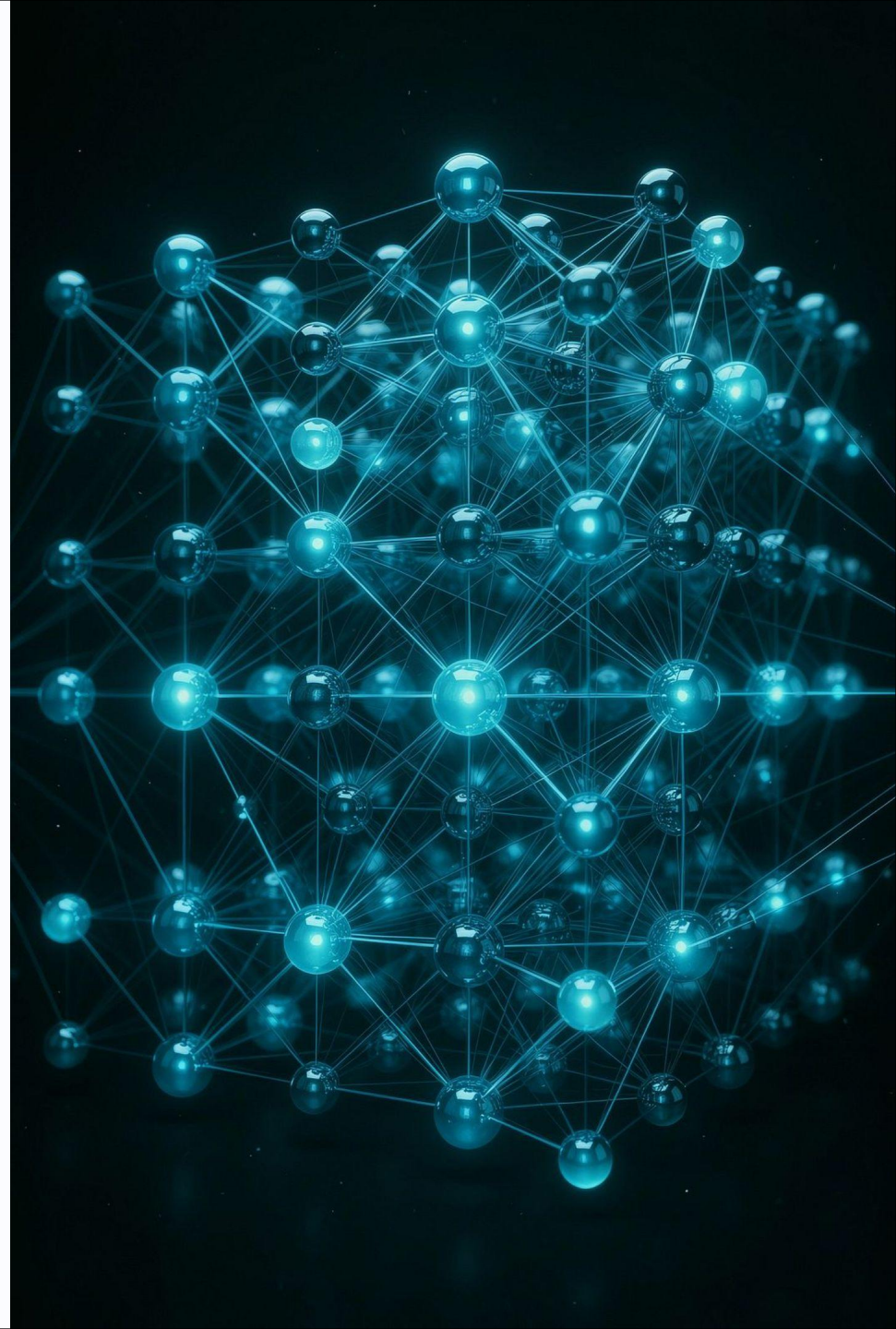
Meta AI — decoder-only Transformer

Gemini

Google DeepMind — multimodal Transformer

Claude

Anthropic — Constitutional AI Transformer



Why Do We Need a Transformer?

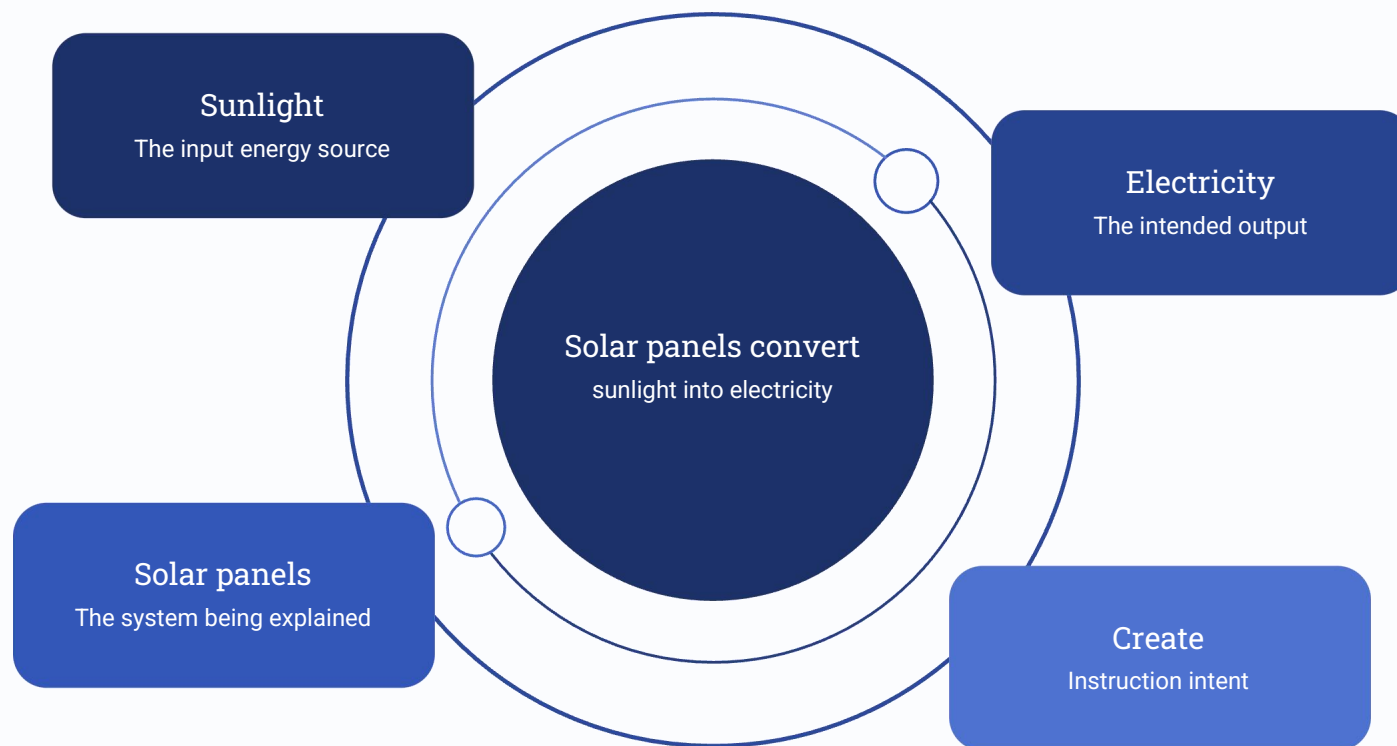
The Challenge of Contextual Understanding

"Explain how solar panels convert sunlight into electricity."

To generate a correct response, the model cannot treat each token in isolation. It must simultaneously understand:

- **Solar panels** – define the subject
- **Convert** – defines the action
- **Sunlight** – defines the input
- **Electricity** – defines the output

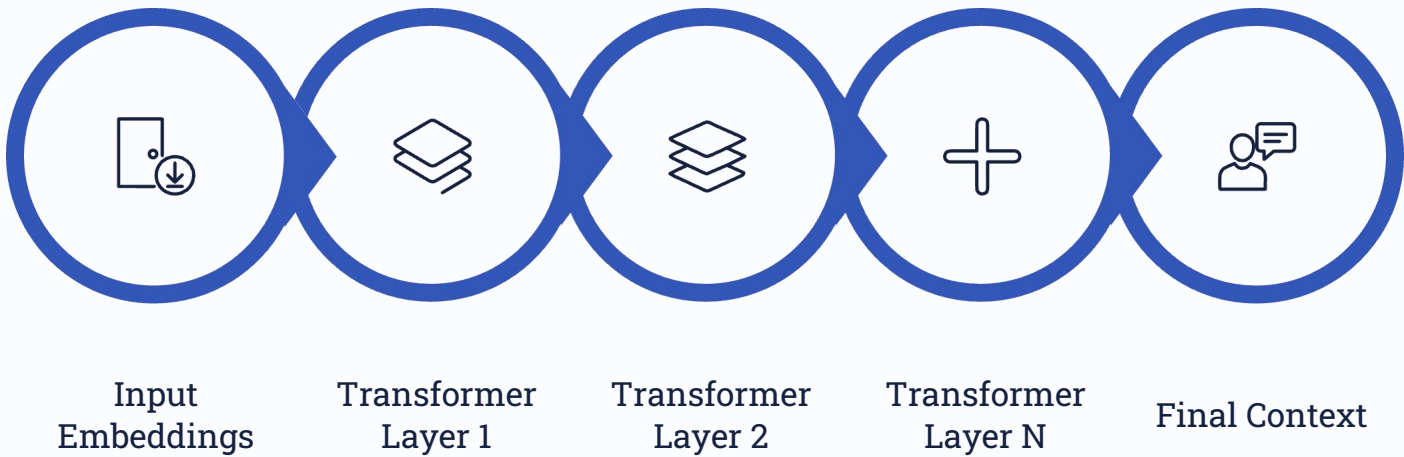
All of these tokens are interdependent. A Transformer processes the full sequence in parallel – computing relationships between every token pair simultaneously.



Transformer Layers

Stacked Processing Architecture

The combined embedding + positional representation passes through a **stack of Transformer Layers**. Each layer refines the contextual representation produced by the previous layer.



Layer Counts by Model Scale

Model	Transformer Layers
GPT-2 (117M)	12
GPT-3 (175B)	96
Llama 2 (7B)	32
Llama 3 (70B)	80

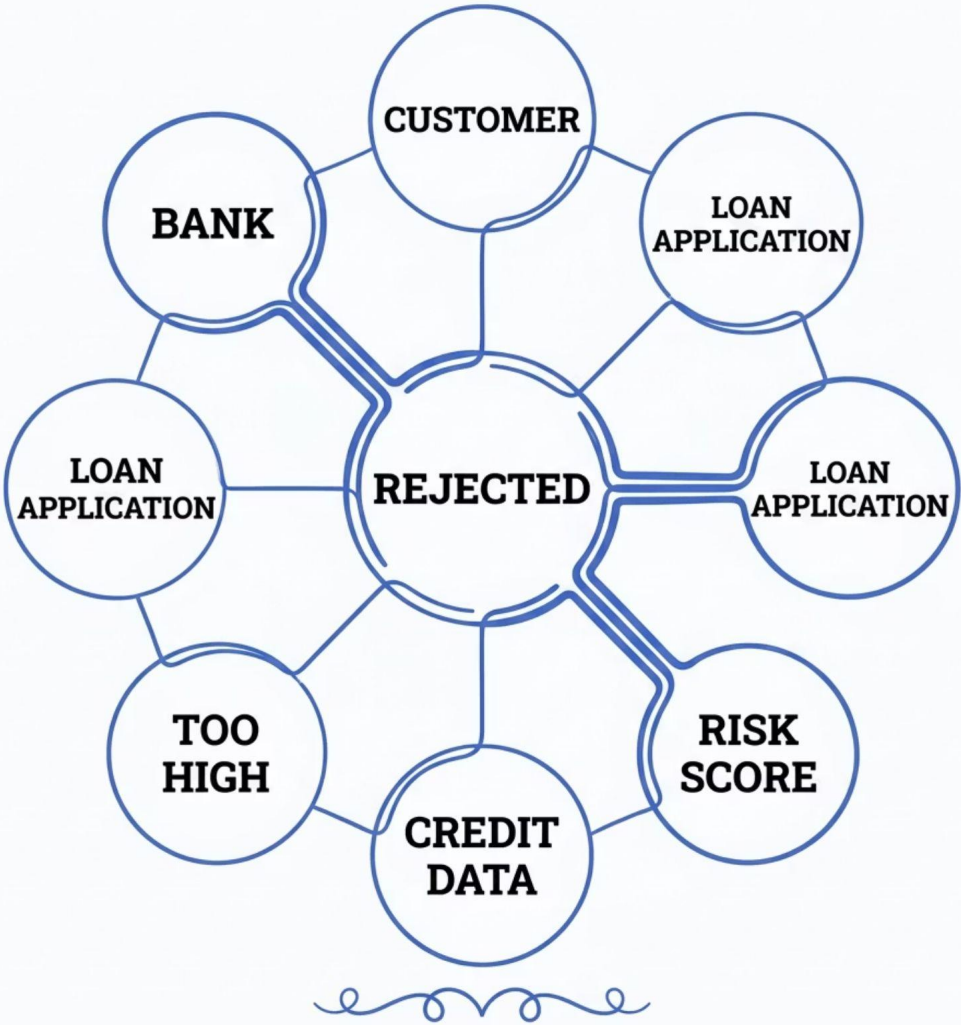
Deeper models with more layers can capture more nuanced linguistic patterns – at the cost of significantly higher compute requirements.

How Does the Model Identify Relationships?

Self-Attention Mechanism

"The bank rejected the customer's loan application because the risk score was too high."

To correctly interpret this sentence, the model must resolve multiple dependencies simultaneously:
Self-Attention allows each token to dynamically attend to all other tokens in the sequence, computing relevance weights that determine how much each token influences the others.



bank → rejected
Subject-action relationship

rejected → loan application
Action-object relationship

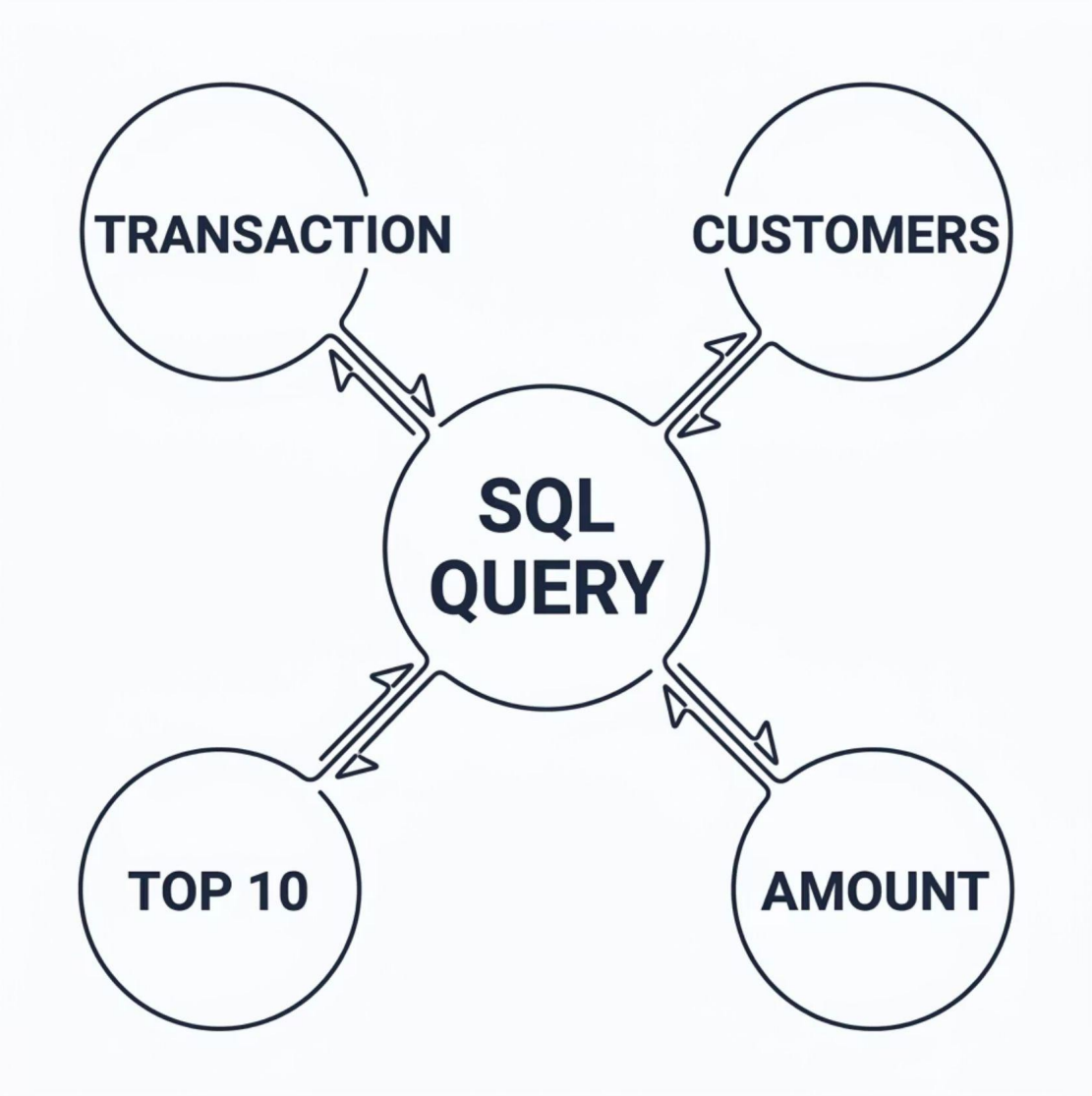
because → risk score
Causal dependency

Self-Attention in Business Context

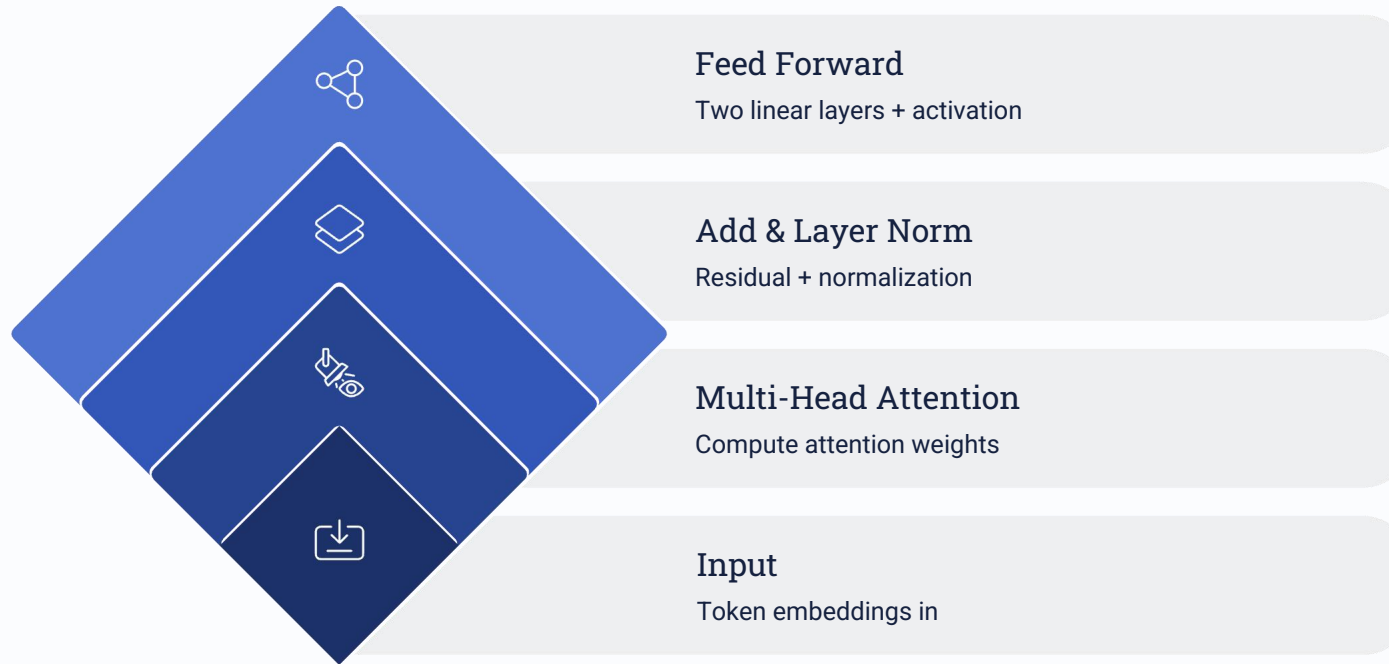
"Create a SQL query to find the top 10 customers based on total transaction amount."

Every token in this prompt is contextually connected to others. The model does not process them sequentially — it computes attention weights across the entire sequence simultaneously.

This is what enables the model to generate **semantically correct, contextually grounded** SQL — rather than a generic template.



Inside a Transformer Layer



Component Breakdown

This complete structure is repeated identically across every layer in the Transformer stack.

1 Multi-Head Self-Attention

Computes attention weights across all token pairs in parallel, across multiple attention heads

2 Add & Layer Normalise

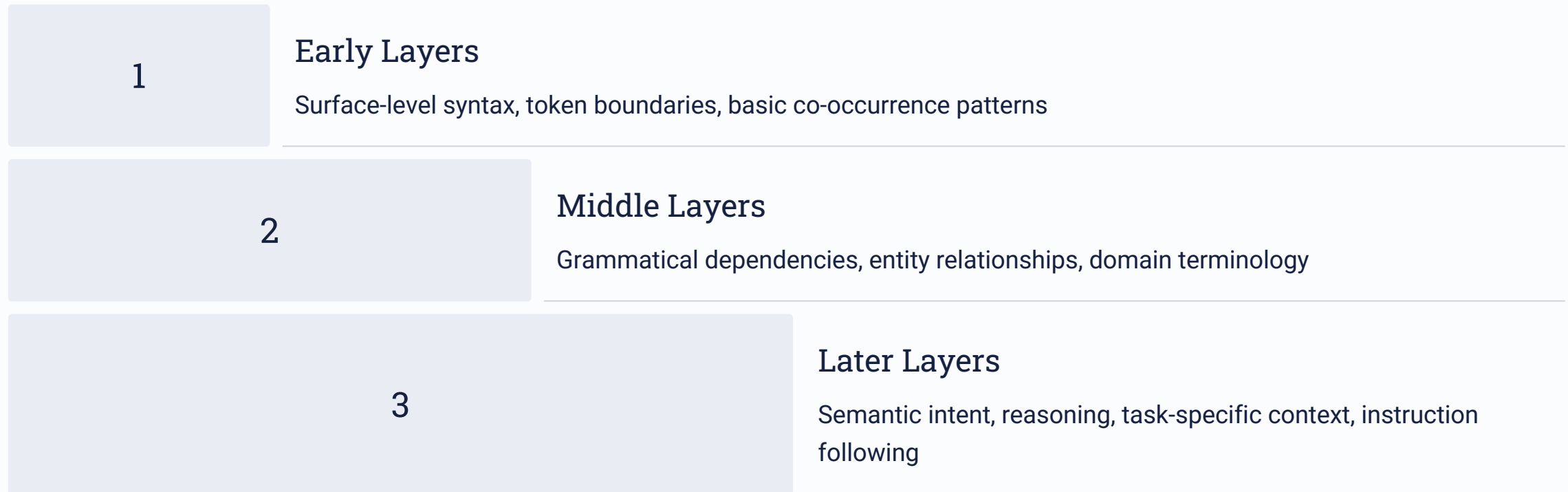
Residual connection adds the input back; layer normalisation stabilises training

3 Feed Forward Network

Position-wise transformation applies learned non-linear projections to each token's representation

What Happens Across Multiple Transformer Layers?

As the representation passes through successive layers, the model builds progressively richer contextual understanding.



By the final layer, the model has constructed a rich, task-aware contextual representation of the complete input — ready for token generation.

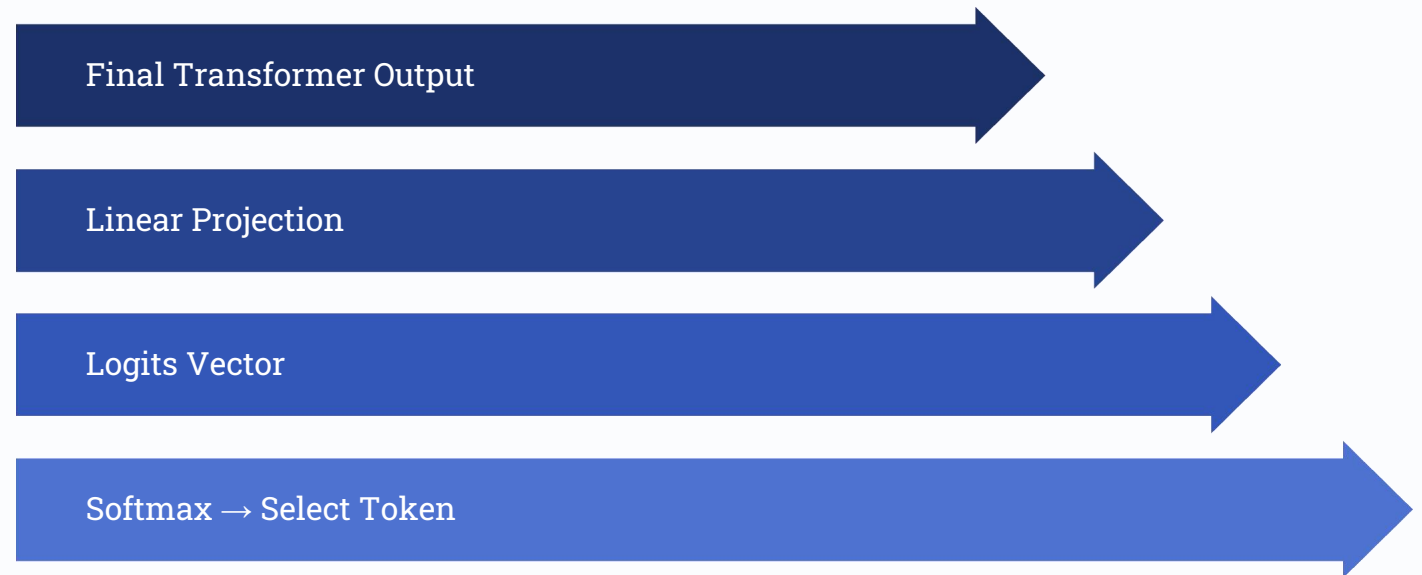
After Transformer Processing

From Understanding to Generation

The model has now processed the full context. It has computed relationships, resolved dependencies, and built a rich representation of the input. But how does it produce the output?

The Transformer's final hidden state is passed to a **Language Model Head** — a linear projection layer that maps the contextual representation to a probability distribution across the model's entire vocabulary.

This drives the next stage: **Next Token Prediction**.



Next Token Prediction

Probability Distribution Over Vocabulary

For each generation step, the model computes a probability score for every token in its vocabulary. The generation strategy then selects the next token.

Candidate Token	Probability
"The"	0.42
"Solar"	0.28
"panels"	0.11
"It"	0.05
All other tokens	0.14 (combined)

Generation Strategies

Greedy Decoding

Always selects the highest probability token

Temperature Sampling

Introduces controlled randomness to vary outputs

Top-K / Top-P

Limits sampling to top candidates by probability mass

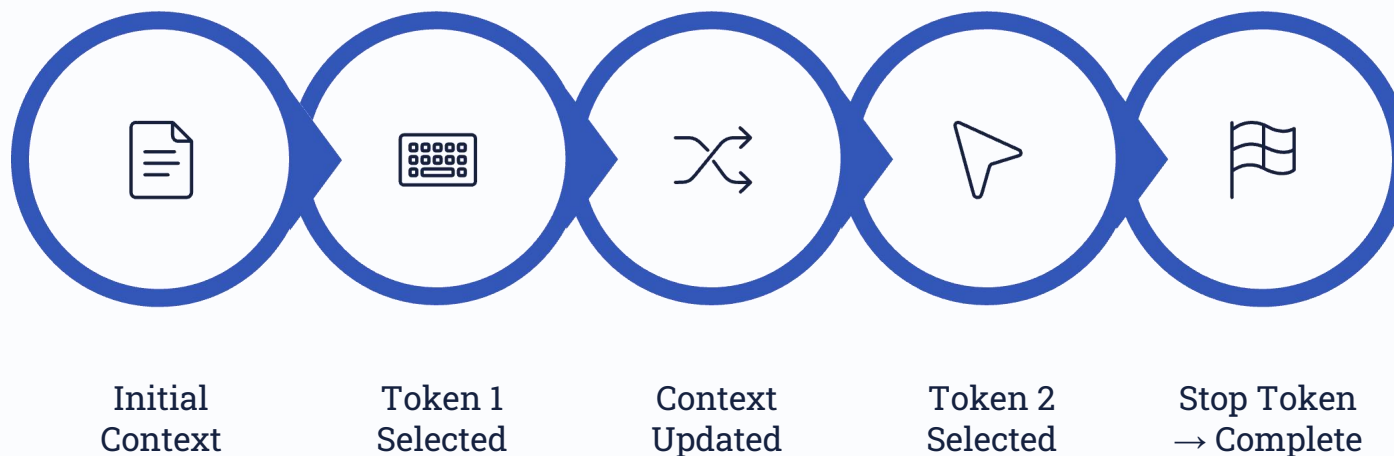
Beam Search

Explores multiple parallel sequences simultaneously

Generating the Complete Response

Autoregressive Generation

LLMs generate output **one token at a time**. After each predicted token is selected, it is appended to the context — and the full pipeline runs again to predict the next token.



Key Characteristics

Sequential by Design

Each token prediction depends on all previously generated tokens

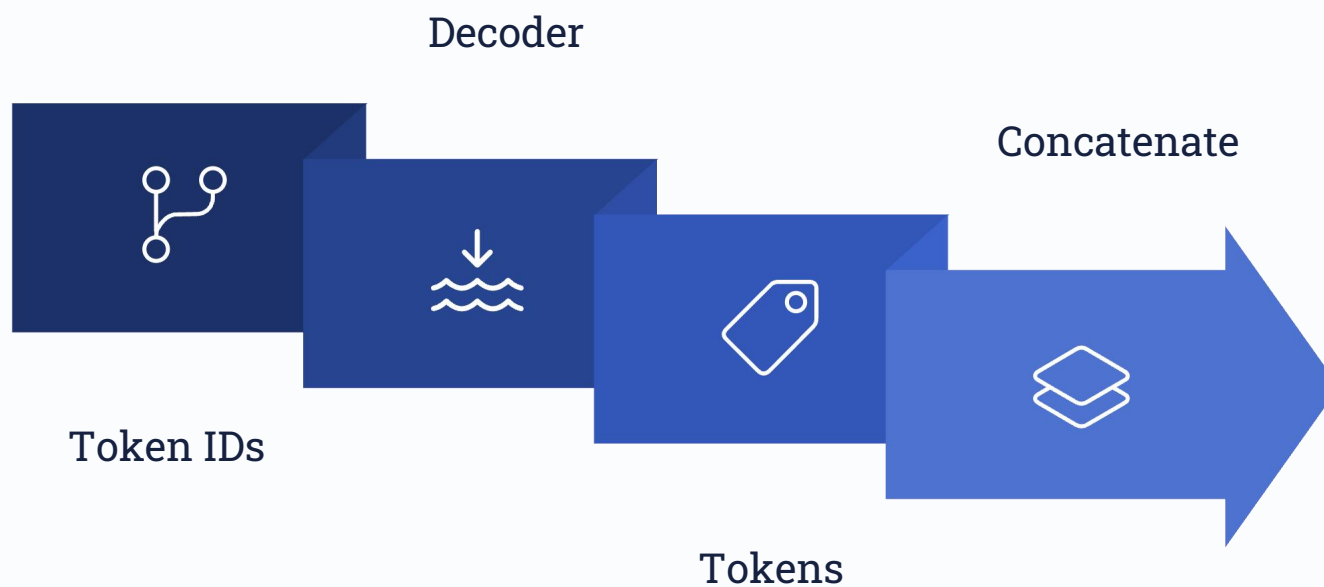
Context Window Constraint

The growing sequence must remain within the model's maximum context window

Termination

Generation stops upon producing a special end-of-sequence token or reaching the max token limit

Detokenisation – Converting Tokens Back to Text



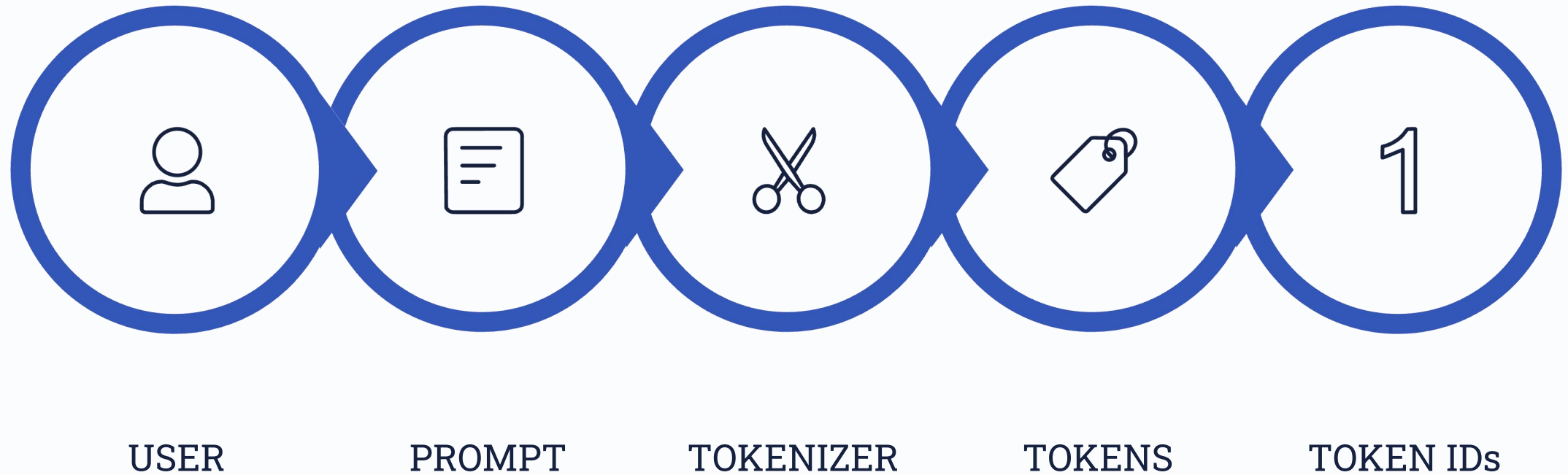
From IDs to Language

Once the model has generated the complete token sequence, the tokeniser's decode operation maps Token IDs back to their corresponding text fragments and assembles them into the final response.

Generated Tokens	Assembled Output
"Solar" + "panels" + "generate" + "electricity"	"Solar panels generate electricity."

- ❏ Detokenisation is the mirror operation of tokenisation – it is deterministic and lossless. The final output exactly represents the generated token sequence.

Complete End-to-End LLM Architecture



- The loop from Next Token Prediction back to the input represents autoregressive generation – the process repeats for every token in the output until the complete response is assembled.

Real-World Example – Document Summarization

"Summarize this operations report in five sentences and retain the key figures."



Tokenisation

Prompt and report text are split into tokens and mapped to IDs



Embeddings

Tokens become learned vectors with position information



Transformer

Attention connects findings, numbers, and qualifications across the report



Generated Summary

A concise draft that must be checked against the source

The same model architecture supports many language tasks. The prompt and supplied context define what the application asks it to produce.

Important Limitations of LLMs

Hallucination – Confident but Incorrect Generation

LLMs do not retrieve facts from a structured database. They generate responses by predicting statistically likely token sequences based on learned patterns. When the training data is insufficient, outdated, or ambiguous, the model can generate factually incorrect content – with high confidence.

Financial Services

Regulatory figures, compliance rules – require validated sources

Healthcare

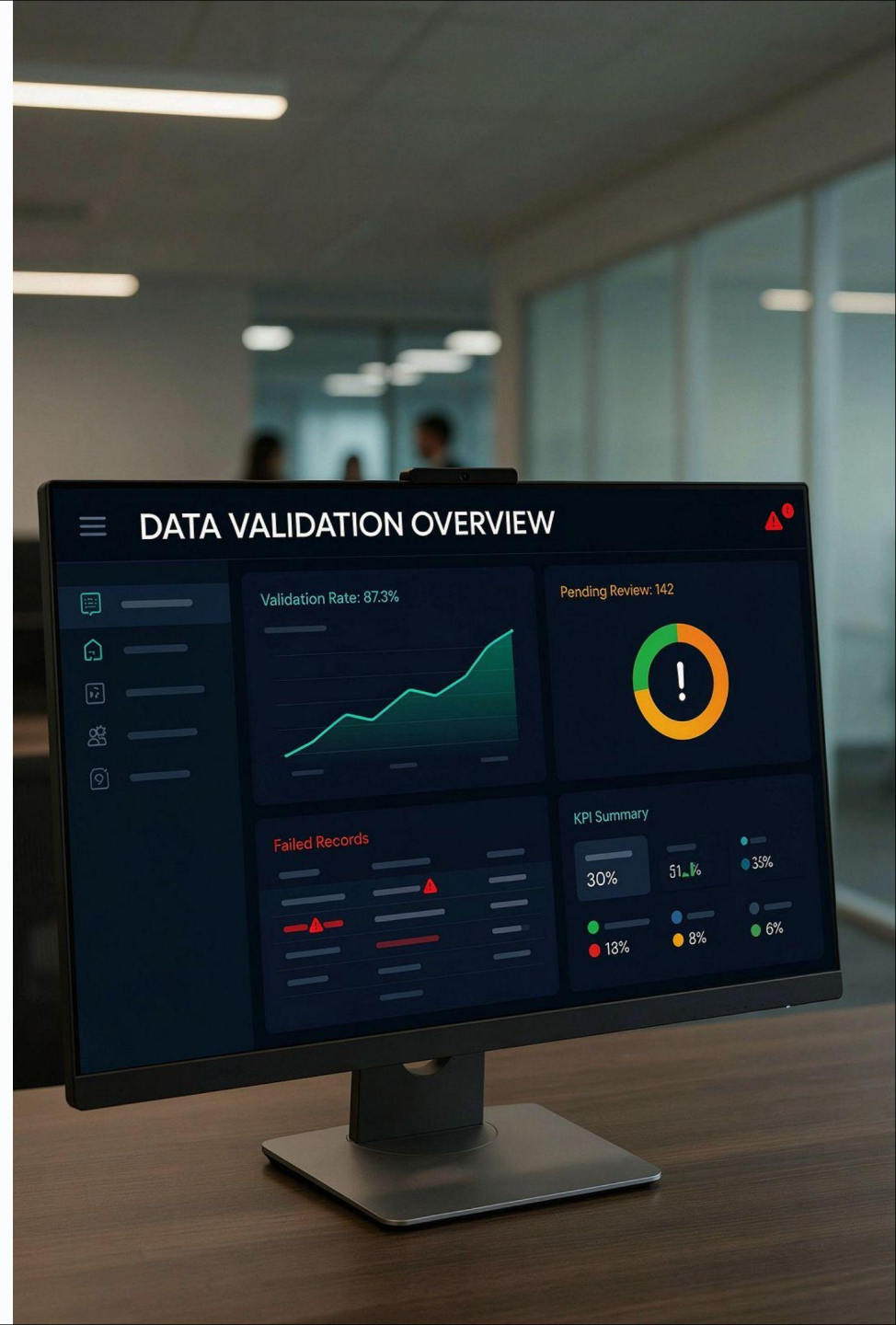
Clinical data, drug interactions – zero tolerance for hallucination

Legal & Audit

Contract terms, case citations – must be verifiable

Enterprise Reporting

KPIs, business metrics – require grounding in actual data



The Role of Prompt Engineering

Input Quality Determines Output Quality

The structure, specificity, and context of the prompt directly influence the model's output relevance and accuracy. Prompt Engineering is the practice of designing inputs that elicit precise, high-quality responses.

Weak Prompt	Engineered Prompt
"Explain cloud."	"Explain cloud computing for a banking organisation, including security considerations, compliance requirements, and migration challenges."

Effective Prompt Components

- 1

Clear Instruction

Specify the task type – generate, summarise, compare, classify
- 2

Relevant Context

Provide domain, constraints, audience, or prior information
- 3

Expected Output Format

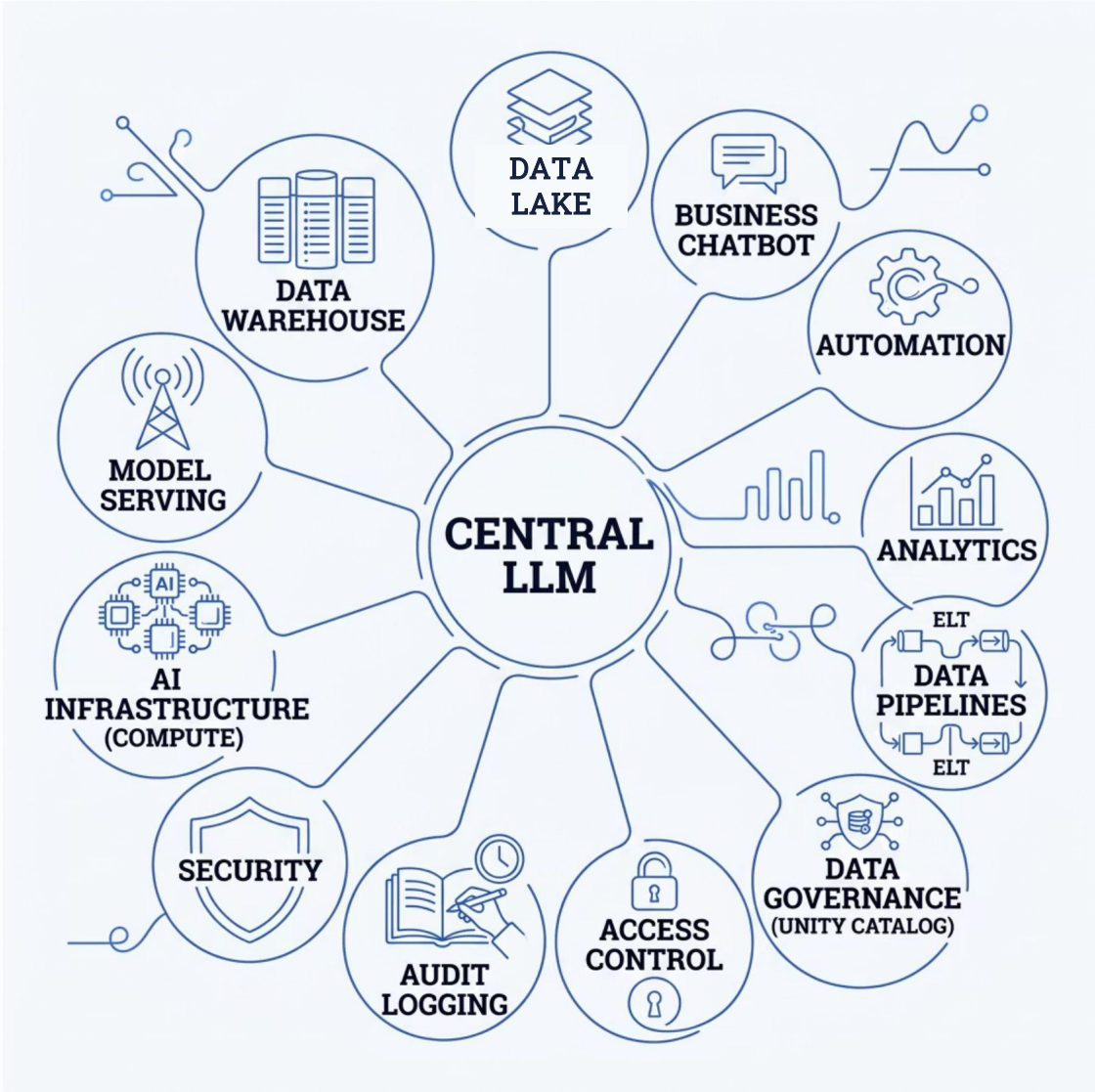
Define structure – JSON, SQL, bullet points, code
- For enterprise use cases, prompt templates are often managed programmatically within data pipelines and AI applications.

Connecting LLMs with Enterprise Data

The Gap Between General LLMs and Enterprise Reality

A foundation LLM is trained on public data. It has no knowledge of your organisation's internal datasets, business rules, customer records, or proprietary schemas.

To build production-grade Generative AI applications, organisations must bridge this gap by integrating LLMs with their enterprise data landscape.



RAG

Retrieval-Augmented Generation — ground responses in live enterprise data

Fine-Tuning

Adapt the model to domain-specific language and tasks

Governance

Data access control, lineage, and compliance enforcement

How LLMs Connect to Enterprise Applications

Production applications surround the model with data access, business rules, security controls, and monitoring. These components determine what the model can see and how the application uses its response.

Application Data

Documents, databases, and live services supply task-specific information

Orchestration

Prompt assembly, retrieval, model calls, and output handling connect the workflow

Security and Governance

Permissions, data handling, audit records, and review protect sensitive use cases

Model Access

Applications can use hosted model APIs or locally managed models



Key Takeaways

The complete journey – from a single user prompt to a coherent, contextually accurate response – involves a sophisticated chain of mathematical transformations:

01	02	03
Prompt → Tokenisation	Token IDs → Embeddings	Embeddings → Transformer Layers
Raw text is split into tokens and mapped to Token IDs via the model's vocabulary	Sparse IDs converted to dense semantic vectors; positional encoding adds sequence information	Multi-head self-attention computes relationships; feed-forward layers refine representations across N layers
04	05	
Context → Next Token Prediction	Token IDs → Detokenisation	
Language model head computes probability distribution; generation strategy selects next token	Generated token sequence decoded back to human-readable text – response delivered	

From Prompt to Response – an LLM performs multiple mathematical transformations in milliseconds to understand context and generate meaningful output.